



UNIVERSIDAD AUTÓNOMA DE CHIAPAS
FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN C-I



LICENCIATURA EN INGENIERIA EN DESARROLLO Y TECNOLOGÍAS DE
SOFTWARE

ALUMNOS:

Yeiner Yair González Pérez

ACTIVIDAD:

Configura MVC en NODE Js usuario con Bd mysql

DOCENTE:

DR. Luis Gutiérrez Alfaro

MATERIA:

Taller de Desarrollo 3

GRUPO:

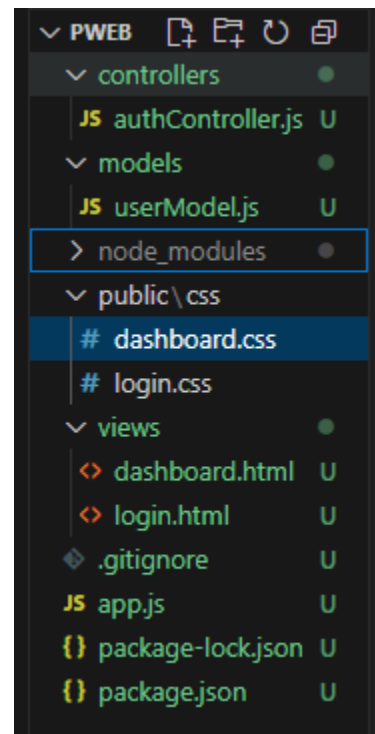
5M

TUXTLA GUTIÉRREZ CHIAPAS A 11 DE SEPTIEMBRE DE 2025

El proyecto sigue la siguiente estructura:

pweb /

```
├── controllers/
│   └── authController.js
│
├── models/
│   └── userModel.js
│
├── views/
│   ├── login.html
│   └── dashboard.html
│
├── public/
│   └── css/
│       ├── login.css
│       └── dashboard.css
│
├── app.js
└── package.json
```



Usaremos Workbench mysql

En Workbench

```
ALTER USER 'root'@'localhost'
```

```
IDENTIFIED WITH mysql_native_password BY '5002y';
```

```
FLUSH PRIVILEGES;
```

```
CREATE DATABASE db_clinica;
```

```
USE db_clinica;
```

```
CREATE TABLE users (
```

```

id INT AUTO_INCREMENT PRIMARY KEY,

username VARCHAR(50),

password VARCHAR(50)

);

INSERT INTO users (username, password) VALUES ('admin', '1234'); */

```

The screenshot shows a MySQL IDE interface. On the left, the 'SCHEMAS' pane displays a tree view of databases, with 'db_clinica' selected and its 'Tables' folder expanded, showing the 'users' table. The main editor window displays an SQL script with line numbers 171 through 186. The script creates a database 'db_clinica', uses it, creates a 'users' table with columns 'id' (auto-increment primary key), 'username' (VARCHAR(50)), and 'password' (VARCHAR(50)), inserts an 'admin' user with password '1234', alters the root user's password, and finally selects all data from the 'users' table. Below the editor, the 'Result Grid' shows the output of the last query, displaying a single row with 'id' 1, 'username' 'admin', and 'password' '1234'. The bottom left pane shows the 'Table: users' structure with columns 'id' (int AI), 'username' (varchar), and 'password' (varchar).

```

171
172
173 CREATE DATABASE db_clinica;
174 USE db_clinica;
175 CREATE TABLE users (
176 id INT AUTO_INCREMENT PRIMARY KEY,
177 username VARCHAR(50),
178 password VARCHAR(50)
179 );
180 INSERT INTO users (username, password) VALUES ('ad
181
182 ALTER USER 'root'@'localhost'
183 IDENTIFIED WITH mysql_native_password BY '5002y';
184 FLUSH PRIVILEGES;
185
186 select* from users;

```

	id	username	password
▶	1	admin	1234
•	NULL	NULL	NULL

Table: users

Columns:

- id int AI
- username varchar
- password varchar

Inicializa el proyecto

npm init -y

```
PS C:\Users\User\OneDrive\Documentos\pweb> npm init -y
Wrote to C:\Users\User\OneDrive\Documentos\pweb\package.json:

{
  "name": "pweb",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^2.2.0",
    "express": "^5.1.0",
    "express-session": "^1.18.2",
    "mysql": "^2.18.1"
  },
  "devDependencies": {},
  "repository": {
    "type": "git",
    "url": "git+https://github.com/GonzalezPerezYeinerYair/Taller3_Examen_100019140.git"
  },
  "bugs": {
    "url": "https://github.com/GonzalezPerezYeinerYair/Taller3_Examen_100019140/issues"
  },
  "homepage": "https://github.com/GonzalezPerezYeinerYair/Taller3_Examen_100019140#readme"
}

PS C:\Users\User\OneDrive\Documentos\pweb> 
```

npm install express mysql body-parser express-session

```
PS C:\Users\User\OneDrive\Documentos\pweb> npm install express mysql body-parser express-session

up to date, audited 88 packages in 5s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

npm install -g nodemon

```
PS C:\Users\User\OneDrive\Documentos\pweb> npm install -g nodemon

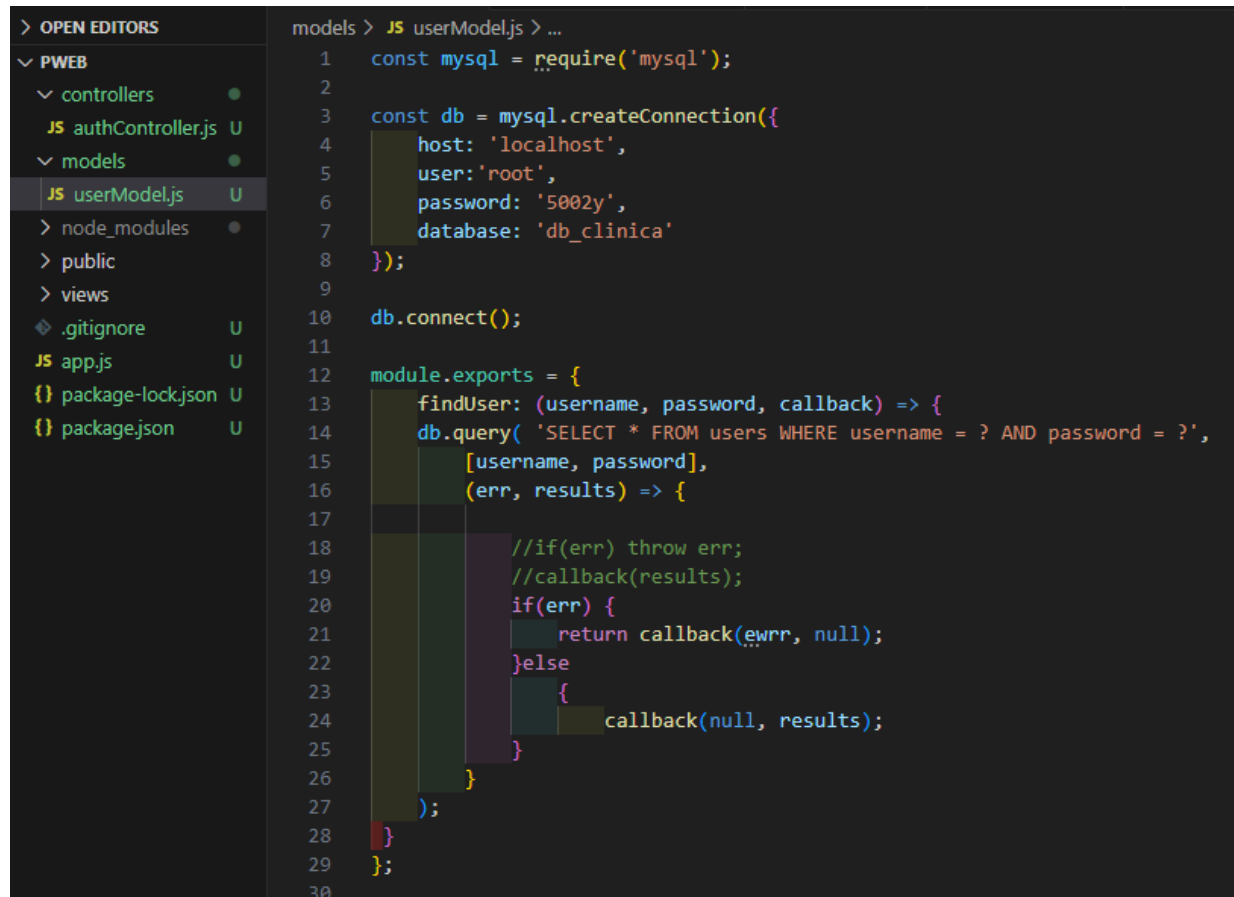
changed 29 packages in 10s

4 packages are looking for funding
  run `npm fund` for details
PS C:\Users\User\OneDrive\Documentos\pweb> 
```

nodemon app.js

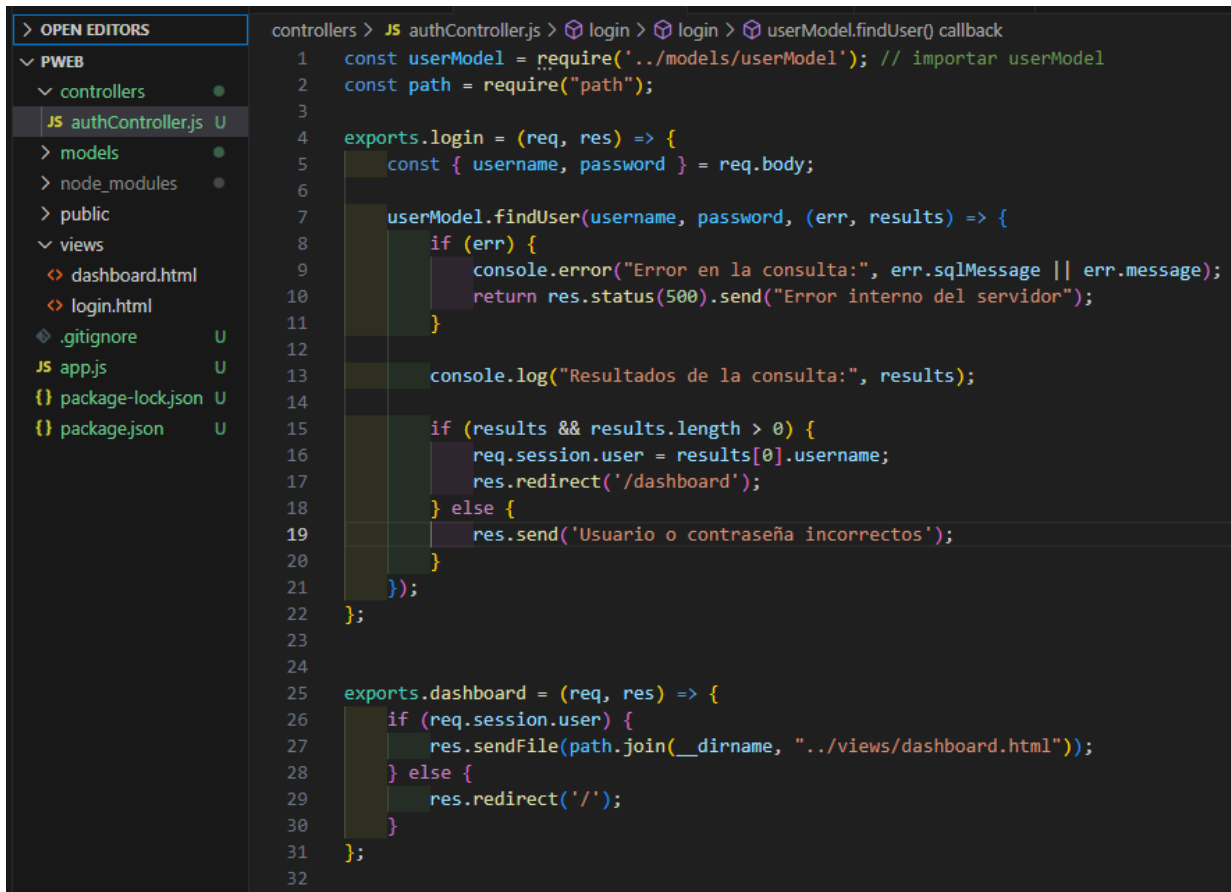
```
PS C:\Users\User\OneDrive\Documentos\pweb> nodemon app.js
[nodemon] 3.1.10
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Servidor corriendo en http://localhost:3000
```

Modelo (models/userModel.js)



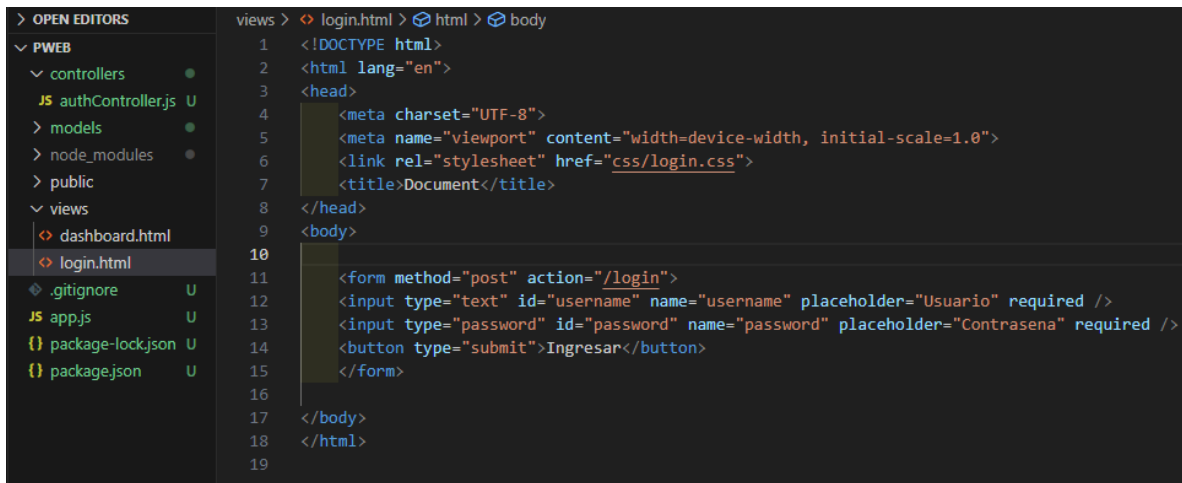
```
models > JS userModel.js > ...
1  const mysql = require('mysql');
2
3  const db = mysql.createConnection({
4    host: 'localhost',
5    user: 'root',
6    password: '5002y',
7    database: 'db_clinica'
8  });
9
10 db.connect();
11
12 module.exports = {
13   findUser: (username, password, callback) => {
14     db.query('SELECT * FROM users WHERE username = ? AND password = ?',
15       [username, password],
16       (err, results) => {
17
18         //if(err) throw err;
19         //callback(results);
20         if(err) {
21           return callback(err, null);
22         } else {
23           {
24             callback(null, results);
25           }
26         }
27       }
28     );
29   }
30 };
```

Controlador (controllers/authController.js)

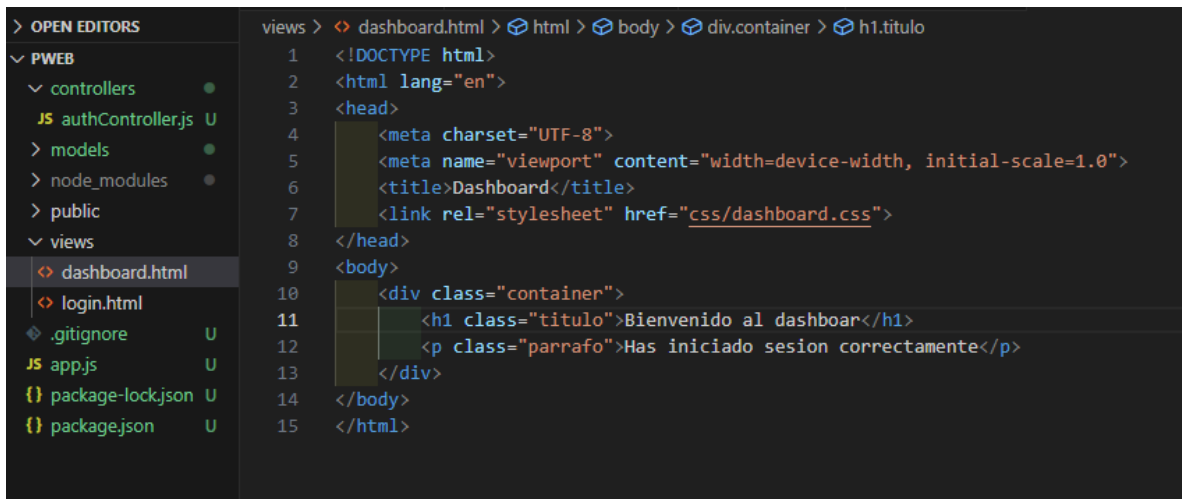


```
controllers > JS authController.js > login > login > userModel.findUser() callback
1  const userModel = require('../models/userModel'); // importar userModel
2  const path = require("path");
3
4  exports.login = (req, res) => {
5      const { username, password } = req.body;
6
7      userModel.findUser(username, password, (err, results) => {
8          if (err) {
9              console.error("Error en la consulta:", err.sqlMessage || err.message);
10             return res.status(500).send("Error interno del servidor");
11         }
12
13         console.log("Resultados de la consulta:", results);
14
15         if (results && results.length > 0) {
16             req.session.user = results[0].username;
17             res.redirect('/dashboard');
18         } else {
19             res.send('Usuario o contraseña incorrectos');
20         }
21     });
22 };
23
24
25 exports.dashboard = (req, res) => {
26     if (req.session.user) {
27         res.sendFile(path.join(__dirname, "../views/dashboard.html"));
28     } else {
29         res.redirect('/');
30     }
31 };
32
```

Vistas (views/login.html y dashboard.html)

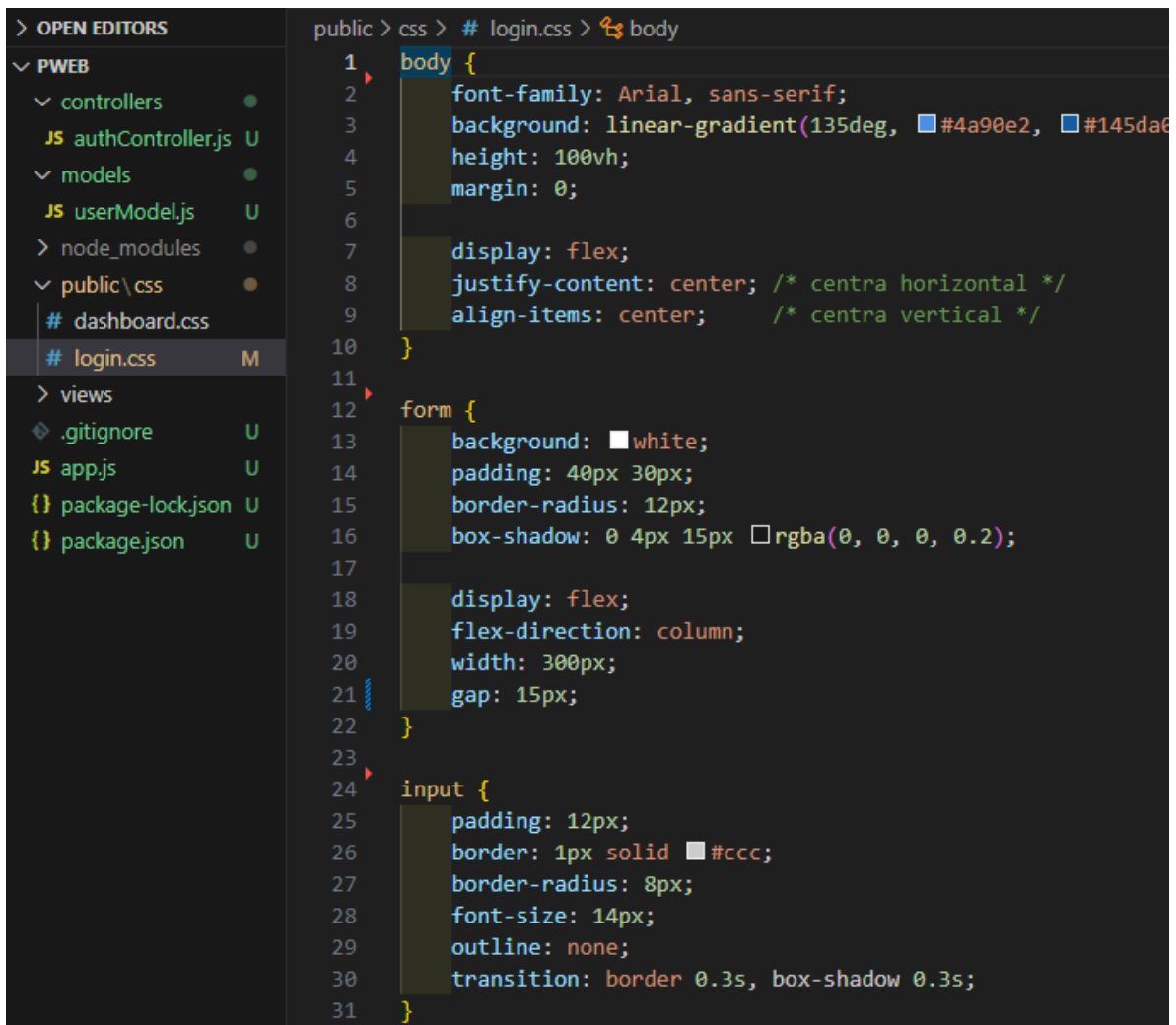


```
views > login.html > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link rel="stylesheet" href="css/login.css">
7   <title>Document</title>
8 </head>
9 <body>
10
11   <form method="post" action="/login">
12     <input type="text" id="username" name="username" placeholder="Usuario" required />
13     <input type="password" id="password" name="password" placeholder="Contraseña" required />
14     <button type="submit">Ingresar</button>
15   </form>
16
17 </body>
18 </html>
19
```



```
views > dashboard.html > html > body > div.container > h1.titulo
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Dashboard</title>
7   <link rel="stylesheet" href="css/dashboard.css">
8 </head>
9 <body>
10   <div class="container">
11     <h1 class="titulo">Bienvenido al dashboar</h1>
12     <p class="parrafo">Has iniciado sesion correctamente</p>
13   </div>
14 </body>
15 </html>
```

Estilos (public/css/login.css dashboard.css)



The image shows a code editor with a sidebar on the left and a main editor area on the right. The sidebar, titled 'OPEN EDITORS', lists several files: 'PWEB' (expanded), 'controllers' (expanded), 'authController.js' (U), 'models' (expanded), 'userModel.js' (U), 'node_modules', 'public\css' (expanded), 'dashboard.css', 'login.css' (M), 'views', '.gitignore' (U), 'app.js' (U), 'package-lock.json' (U), and 'package.json' (U). The main editor area shows the content of 'login.css' with the following CSS code:

```
public > css > # login.css > body
1 body {
2   font-family: Arial, sans-serif;
3   background: linear-gradient(135deg, #4a90e2, #145da6);
4   height: 100vh;
5   margin: 0;
6
7   display: flex;
8   justify-content: center; /* centra horizontal */
9   align-items: center;     /* centra vertical */
10 }
11
12 form {
13   background: white;
14   padding: 40px 30px;
15   border-radius: 12px;
16   box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);
17
18   display: flex;
19   flex-direction: column;
20   width: 300px;
21   gap: 15px;
22 }
23
24 input {
25   padding: 12px;
26   border: 1px solid #ccc;
27   border-radius: 8px;
28   font-size: 14px;
29   outline: none;
30   transition: border 0.3s, box-shadow 0.3s;
31 }
```


> OPEN EDITORS

✓ PWEB

controllers

JS authController.js U

models

JS userModel.js U

node_modules

public\css

dashboard.css

login.css M

views

.gitignore U

JS app.js U

{ } package-lock.json U

{ } package.json U

public > css > # login.css > body

22 }

23

24 input {

25 padding: 12px;

26 border: 1px solid #ccc;

27 border-radius: 8px;

28 font-size: 14px;

29 outline: none;

30 transition: border 0.3s, box-shadow 0.3s;

31 }

32

33 input:focus {

34 border-color: #4a90e2;

35 box-shadow: 0 0 5px rgba(74, 144, 226, 0.6);

36 }

37

38 button {

39 padding: 12px;

40 background: #4a90e2;

41 color: white;

42 border: none;

43 border-radius: 8px;

44 font-size: 16px;

45 font-weight: bold;

46 cursor: pointer;

47 transition: background 0.3s;

48 }

49

50 button:hover {

51 background: rgb(61, 124, 61);

52 }

53

> OPEN EDITORS

▼ PWEB

▼ controllers

JS authController.js U

▼ models

JS userModel.js U

> node_modules

▼ public\css

dashboard.css M

login.css M

> views

◆ .gitignore

U

JS app.js U

{ } package-lock.json U

{ } package.json U

public > css > # dashboard.css > body

1

▼ body {

2

font-family: Arial, sans-serif;

3

background-color: rgb(64, 65, 63);

4

display: flex;

5

justify-content: center; /* centra horizontal */

6

align-items: center; /* centra vertical */

7

height: 100vh;

8

margin: 0;

9

}

10

▼ .titulo {

11

text-align: center;

12

color: red;

13

margin-top: 20px;

14

font: bold 24px Arial, sans-serif;

15

}

16

17

▼ .parrafo {

18

text-align: center;

19

color: black;

20

margin-top: 10px;

21

}

22

23

▼ .container {

24

background: white;

25

padding: 30px;

26

border-radius: 12px;

27

box-shadow: 0 4px 10px rgba(0,0,0,0.2);

28

text-align: center;

29

}

30

31

▼ .container:hover {

32

transition: all 0.5s ease;

33

background-color: rgb(19, 72, 141);

34

box-shadow: 0 6px 12px rgba(42, 44, 122, 0.3);

35

▼ .titulo {

36

color: white;

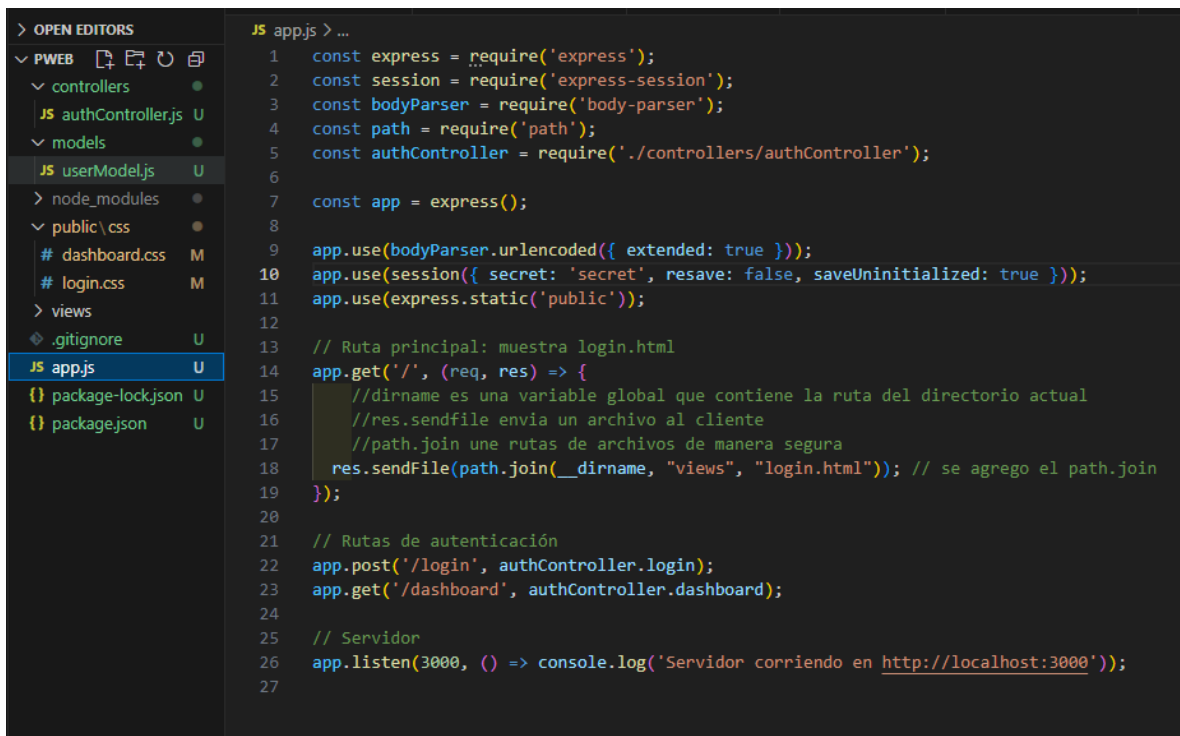
37

}

38

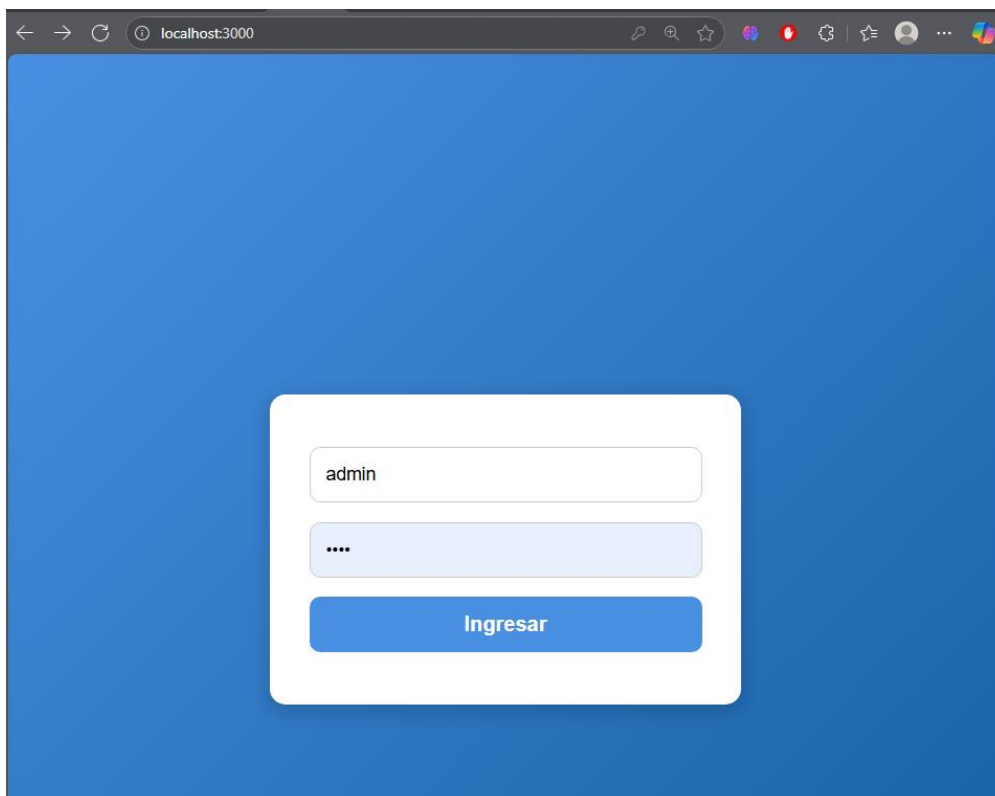
}

Servidor principal (app.js)

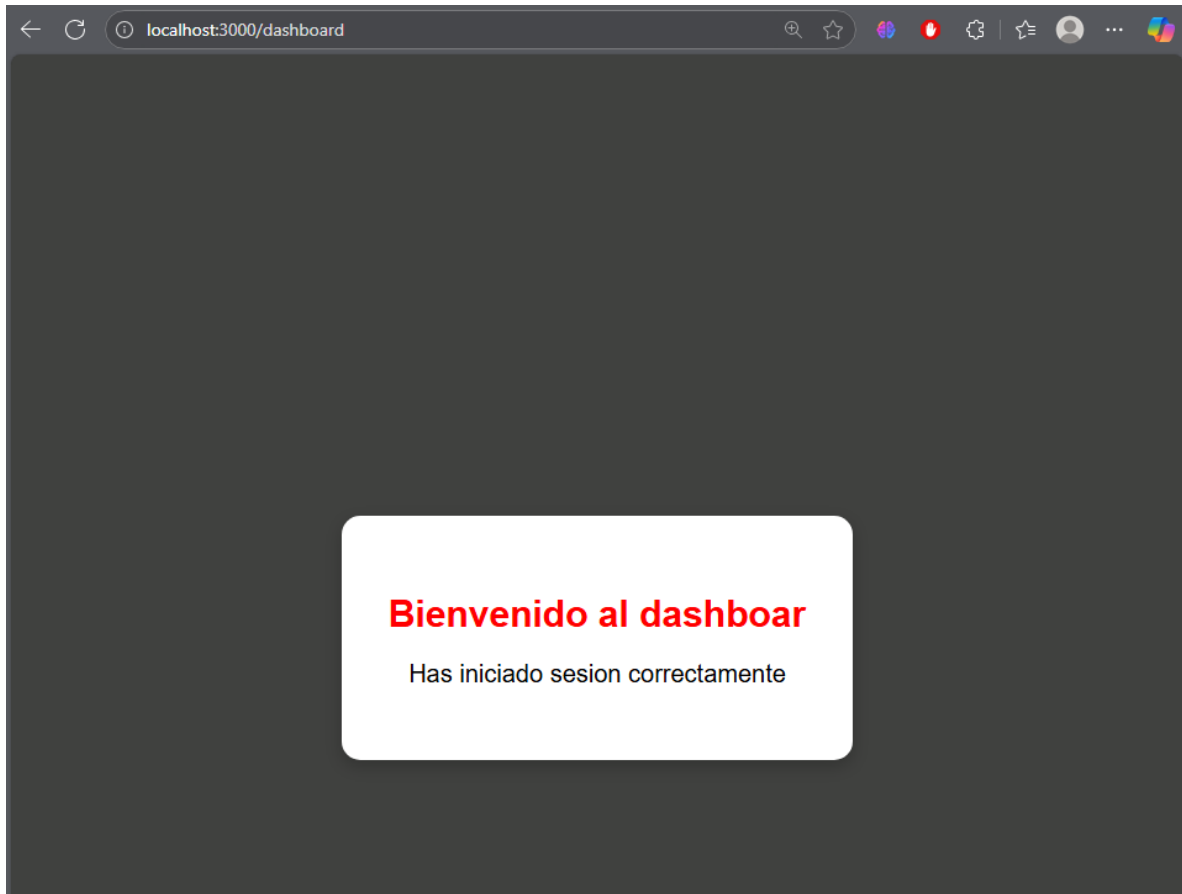


```
JS app.js > ...
1  const express = require('express');
2  const session = require('express-session');
3  const bodyParser = require('body-parser');
4  const path = require('path');
5  const authController = require('./controllers/authController');
6
7  const app = express();
8
9  app.use(bodyParser.urlencoded({ extended: true }));
10 app.use(session({ secret: 'secret', resave: false, saveUninitialized: true }));
11 app.use(express.static('public'));
12
13 // Ruta principal: muestra login.html
14 app.get('/', (req, res) => {
15     //dirname es una variable global que contiene la ruta del directorio actual
16     //res.sendFile envia un archivo al cliente
17     //path.join une rutas de archivos de manera segura
18     res.sendFile(path.join(__dirname, "views", "login.html")); // se agrego el path.join
19 });
20
21 // Rutas de autenticación
22 app.post('/login', authController.login);
23 app.get('/dashboard', authController.dashboard);
24
25 // Servidor
26 app.listen(3000, () => console.log('Servidor corriendo en http://localhost:3000'));
27
```

Login (en el puerto 3000 del servidor)



Login (en el puerto 3000 del servidor) en caso de ingresar con el usuario y contraseña correcta (admin, 1234):



Login (en el puerto 3000 del servidor) en caso de ingresar con el usuario y/o contraseña INCORRECTA:

